Testing the project

Study guide 6: the three tests.py files, tiny data, the test client, fakes, and checking that tests notice

What this guide covers

File	Tests	What they protect
`commission/tests.py`	6	The commission rules (3) and the pages (3)
`activity/tests.py`	5	The activity rules (3) and the pages (2)
`core/tests.py`	8	Months, Excel, the login and home pages, ledger slices

How to use it

- Read Part 1 for the thinking: what deserves a test and how small the data should be.
- Part 3 builds commission/tests.py step by step, then shows the tools the other two files add.
- Do Part 5's mutation exercise: break the code on purpose and watch which test catches it.

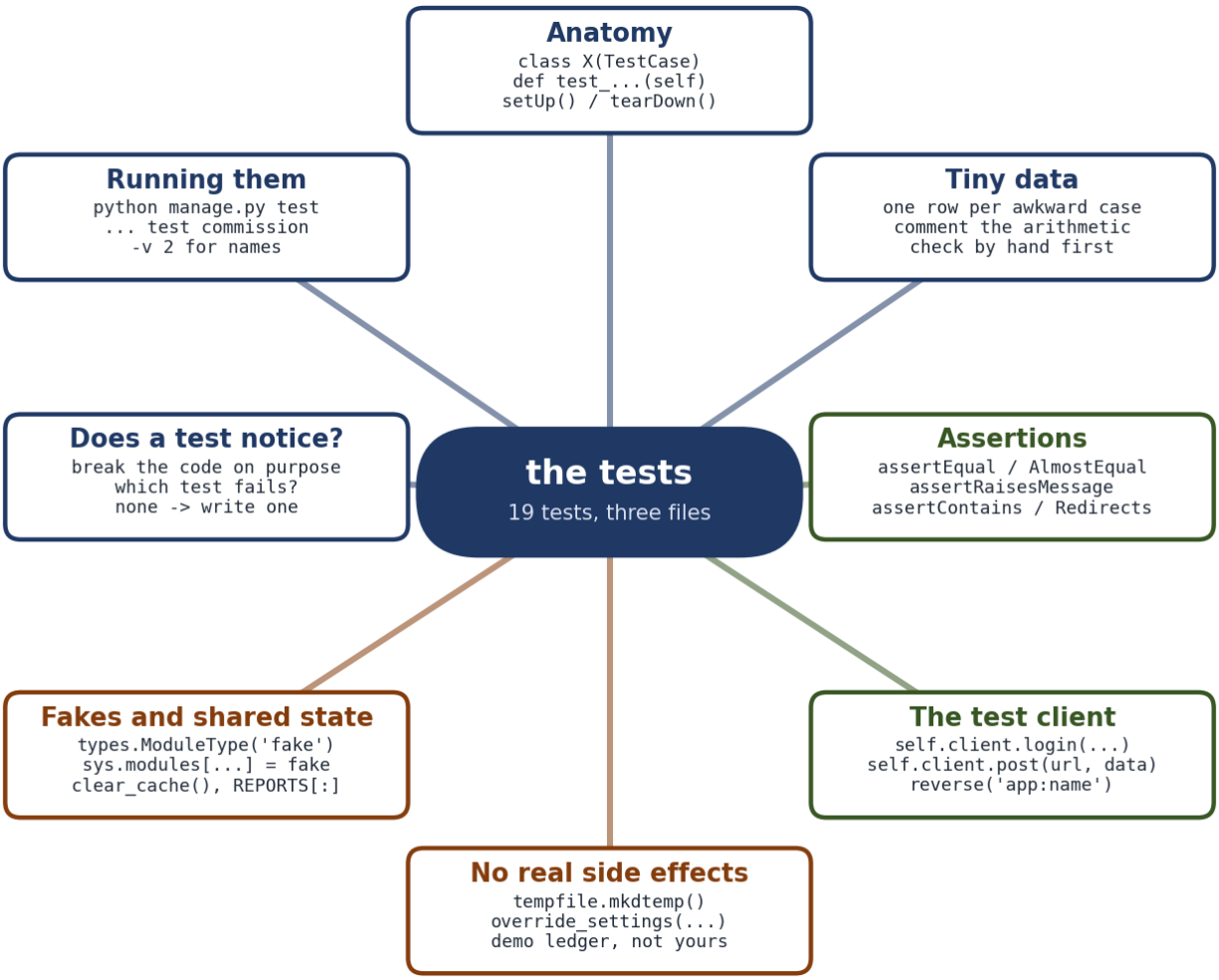
Part 1. Think before you write tests

A test is a small program that runs your code on a case you understand and checks the answer. Its value is simple: **if a rule breaks, you find out before a payout does**. Three questions decide what to write:

1. **Which rule would hurt most if it silently changed?** The month boundary, the NBP rule, who counts as traded. Those get tests first.
2. **What's the smallest table that shows the rule?** Five rows you can add up in your head beat five thousand you can't.
3. **What should happen when the input is wrong?** A clear error is behaviour too; test it.

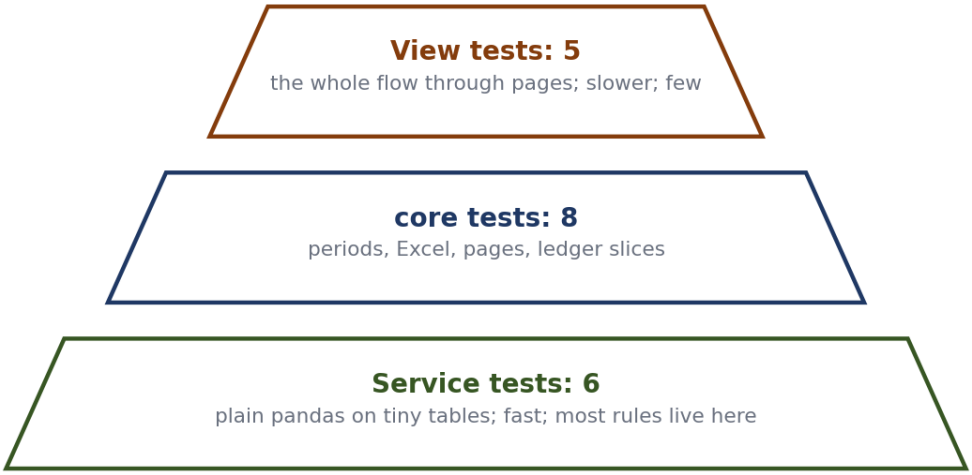
Fact	So the tests...	Tool
Most rules live in services.py, which needs no Django	test those directly on tiny DataFrames	`TestCase / SimpleTestCase`
The pages wire many parts together	run the whole flow once or twice per app	`self.client`
Reports write files	write them to a temporary folder	`tempfile + override_settings`
The real ledger is huge and private	use the demo ledger or a fake	`LEDGER_MODULE, types.ModuleType`
Some state lives at module level	reset it after each test	`tearDown, clear_cache()`
A test that never fails proves nothing	are checked by breaking the code on purpose	`mutation exercise`

All the testing ideas as a mindmap


Anatomy
class X(TestCase)
def test_...(self)
setUp() / tearDown()
Running them
python manage.py test
... test commission
-v 2 for names
Tiny data
one row per awkward case
comment the arithmetic
check by hand first
Does a test notice?
break the code on purpose
which test fails?
none -> write one
the tests
19 tests, three files
Assertions
assertEqual / AlmostEqual
assertRaisesMessage
assertContains / Redirects
Fakes and shared state
types.ModuleType('fake')
sys.modules[...] = fake
clear_cache(), REPORTS[:]
The test client
self.client.login(...)
self.client.post(url, data)
reverse('app:name')
No real side effects
tempfile.mkdtemp()
override_settings(...)
demo ledger, not yours

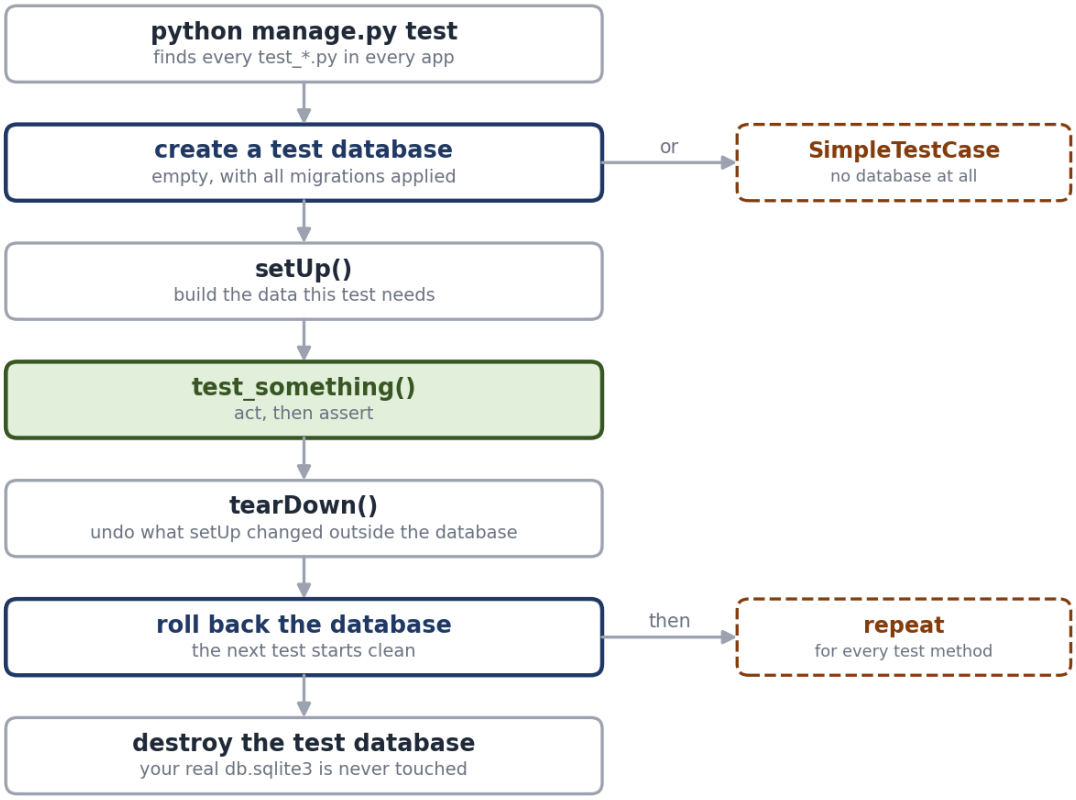
Part 2. The big picture

Where the 19 tests sit



Most rules are checked where they live, in services.py, on tables you can verify by hand.

What happens when you run them



Part 3. Build the tests step by step

Step 1. The shape of a test file

What I was thinking
"A test file is a module named test*.py inside an app. Django's runner finds classes that inherit TestCase and runs every method whose name starts with test_."
"Group tests by what they check: the calculation in one class, the pages in another."
"setUp runs before every test method, so each test gets fresh data."

```
import pandas as pd
from django.test import TestCase

from . import services


class ServiceTests(TestCase):
    """The calculation alone, on a table small enough to check by hand."""

    def setUp(self):
        self.led = pd.DataFrame({...})          # Step 2
        self.brokers = pd.DataFrame({...})

    def test_client_pnl(self):
        ...                                     # Step 3


# in a terminal, from the project folder:
#   python manage.py test                           every app
#   python manage.py test commission                one app
#   python manage.py test commission.tests.ServiceTests.test_client_pnl     one test
#   python manage.py test -v 2                      print each test's name
```

Tool	What it does	Why here
`class X(TestCase)`	A group of tests	Gets assertions, the test client and a test database
`def test_...(self)`	One test	Only methods starting with test_ run
`def setUp(self)`	Runs before each test method	Every test starts from the same data
`self.led = ...`	Stores data on the test object	Reachable from every test method
`python manage.py test app.module.Class.method`	Runs just one test	Fast while fixing one thing

Check yourself
python manage.py test -v 2 lists 19 test names, each followed by '... ok'.

Step 2. Tiny data with every awkward case

What I was thinking
"Each row is there for a reason. If I can't say what a row tests, it doesn't belong."
"Real data has messy types, blank amounts and rows outside the month, so the tiny table has one of each."
"Pick numbers that are easy to add up: 100, 30, 10, 80."

```
    def setUp(self):
        self.led = pd.DataFrame({
            "ClientID":   ["A", "A", "A", "B", "B"],
            "CreatedAt":  pd.to_datetime(["2026-08-02", "2026-08-03", "2026-08-04", "2026-08-05", "2026-09-01"]),
            "LedgerType": ["BILL", "bill ", "NEGATIVE BALANCE PROTECTION", "BILL", "BILL"],
            "Debit":      [100.0, None, 0.0, 0.0, 500.0],
            "Credit":     [0.0, 30.0, 10.0, 80.0, 0.0],
        })
        self.brokers = pd.DataFrame({"Broker Name": ["X", "X", "Y"], "Broker Client ID": ["P1", "P1", "P2"],
                                     "Broker Referral Count": [2, 2, 1],
                                     "Referred User Client ID": ["A", "B", "C"]})
```

Tool	What it does	Why here
`'bill ' (row 2)`	A messy ledger type	Fails if types aren't stripped and upper-cased
`None in Debit (row 2)`	A blank amount	Fails if blanks aren't treated as 0
`the NBP row (row 3)`	A second ledger type	Fails if NBP is dropped (the 4 Nov bug)
`2026-09-01 (row 5)`	Exactly the end of the range	Fails if the range uses <= end
`client C`	Referred but never traded	Fails if non-traders vanish

Check yourself
For each row, say out loud which bug it would catch.

Step 3. Assertions, with the arithmetic beside them

What I was thinking
"Write the expected number as a calculation in a comment. When a test fails months later, the comment explains what it meant."
"Money from multiplication can be 2.9999999 instead of 3.0, so compare floats with assertAlmostEqual."
"Index the result by ID (set_index) so each assertion names the client it's about."

```
    def test_client_pnl(self):
        start, end = services.month_range("2026-08")
        pnl = services.client_pnl(self.led, ["A", "B"], start, end).set_index("ClientID")
        self.assertEqual(pnl.loc["A", "Net_Profit"], -60)          # 30 - 100 + 10
        self.assertAlmostEqual(pnl.loc["A", "Commission"], 3.0)    # 5% of 60
        self.assertEqual(pnl.loc["B", "Net_Profit"], 80)           # the September loss is outside August

    def test_broker_sheet(self):
        report = services.build_commission(self.led, self.brokers, "2026-08")
        brokers = report["brokers"].set_index("Parent")
        self.assertEqual(brokers.loc["P1", "Difference"], 10)      # (30 + 80) - 100
        self.assertEqual(brokers.loc["P1", "Loss Status"], "NO LOSS")
        self.assertEqual(brokers.loc["P2", "No of Children"], 1)   # C never traded: zeros, still listed
        self.assertEqual(report["summary"]["clients"], 3)
```

Tool	What it does	Why here
`self.assertEqual(a, b)`	Fails if a != b, showing both	Exact values and labels
`self.assertAlmostEqual(a, b)`	Equal to 7 decimal places	Floats from multiplication
`.set_index('ClientID').loc['A', col]`	One cell by ID and column	Readable, order-independent
`# 30 - 100 + 10`	The expected value's derivation	Explains the number to future you

Check yourself
Change 'bill ' to 'BILL' in the data: both tests still pass. Now the messy-type case isn't tested, which is why the data keeps it messy.

Step 4. Test the failure path too

What I was thinking
"A clear error message is a promise to the user. If someone later replaces it with a KeyError, a test should notice."
"assertRaisesMessage checks the exception type and part of its text."

```
    def test_missing_column_is_a_clear_error(self):
        with self.assertRaisesMessage(ValueError, "missing columns"):
            services.client_pnl(self.led.drop(columns="Credit"), ["A"], *services.month_range("2026-08"))
```

Tool	What it does	Why here
`with self.assertRaisesMessage(E, text):`	Passes only if the block raises E with text in its message	The error is part of the contract
`self.led.drop(columns='Credit')`	A copy without that column	The broken input, made from the good one
`*services.month_range(...)`	Unpacks (start, end) into two arguments	f(*pair) is f(pair[0], pair[1])

Check yourself
Change the message in services.py to 'columns not found': this test fails and tells you exactly why.

Step 5. View tests with the test client

What I was thinking
"A view test plays the browser: log in, post the form with a file, follow the redirect, read the page and the download."
"reverse() builds URLs from names, so tests don't break when an address changes."
"Check what the user would notice: the redirect, the saved run, the page text, the download's file name."

```
    def test_create_report(self):
        self.client.login(username="analyst", password="pass12345")
        upload = SimpleUploadedFile("brokers.csv", SAMPLE_CSV.read_bytes(), content_type="text/csv")
        response = self.client.post(reverse("commission:run_create"),
                                    {"month": "2026-08", "rate": "0.05", "broker_file": upload})
        run = CommissionRun.objects.get()
        self.assertRedirects(response, reverse("commission:run_detail", args=[run.pk]))
        self.assertEqual(run.brokers_count, 5)
        self.assertEqual(run.clients_count, 30)
        self.assertTrue(run.result_file.name.endswith(".xlsx"))

        detail = self.client.get(reverse("commission:run_detail", args=[run.pk]))
        self.assertContains(detail, "Demo Broker 1")
        download = self.client.get(reverse("commission:run_download", args=[run.pk]))
        self.assertEqual(download["Content-Disposition"], 'attachment; filename="Broker_Commission_2026-08.xlsx"')

    def test_login_required(self):
        response = self.client.get(reverse("commission:run_list"))
        self.assertEqual(response.status_code, 302)
        self.assertIn(reverse("login"), response["Location"])
```

Tool	What it does	Why here
`self.client.login(username=, password=)`	Logs the fake browser in	Views need a user
`SimpleUploadedFile(name, bytes, content_type)`	An upload made in memory	Posted like a real file
`self.client.post(url, {...})`	Submits the form	Files go in the same dict
`Model.objects.get()`	The only row (errors if 0 or 2+)	Also proves exactly one run was saved
`assertRedirects(response, url)`	Checks the 302 and its target	Post / redirect / get
`assertContains(response, text)`	Status 200 and the text in the page	What the user would see
`response['Content-Disposition']`	A response header	The download's file name

Check yourself
Comment out @login_required on run_list: test_login_required fails with 200 != 302.

Step 6. Keep tests away from real files and real data

What I was thinking
"The report saves files. Tests must never write into the real media folder, so each test gets a temporary folder."
"Point the ledger at the demo, so tests run anywhere and never read client data."
"override_settings(...).enable() in setUp needs a matching .disable() in tearDown, or the change leaks into other tests."

```
class ViewTests(TestCase):
    def setUp(self):
        self.media = tempfile.mkdtemp()
        self.override = override_settings(MEDIA_ROOT=self.media, LEDGER_MODULE="core.demo_ledger")
        self.override.enable()
        self.user = get_user_model().objects.create_user("analyst", password="pass12345")

    def tearDown(self):
        self.override.disable()
        shutil.rmtree(self.media, ignore_errors=True)
```

Tool	What it does	Why here
`tempfile.mkdtemp()`	Creates a new empty folder, returns its path	A private media folder per test
`override_settings(NAME=value)`	Temporarily changes settings	Media and ledger for tests only
`.enable() / .disable()`	Turns the override on and off	Must be paired
`get_user_model().objects.create_user(...)`	A user in the test database	Rolled back after the test
`shutil.rmtree(path, ignore_errors=True)`	Deletes the folder and its files	No leftovers
`with self.settings(NAME=value):`	The same override as a with block	Used in core/tests.py for one call

Check yourself
After python manage.py test, the project's media/ folder has no new files.

Step 7. Fakes and shared state

What I was thinking
"To test the ledger's fallback path I need a loader without scan_ledger. Rather than write a file, build a module object in the test and register it where import looks first: sys.modules."
"Module-level things outlive a test: the ledger cache, the fake module, the REPORTS list. Reset each one in tearDown or a finally block."

```
import sys
import types


@override_settings(LEDGER_MODULE="core.demo_ledger")
class LedgerSliceTests(TestCase):
    def tearDown(self):
        ledger.clear_cache()                        # the cached full ledger
        sys.modules.pop("fake_loader", None)        # the fake module

    def test_loader_without_scan_ledger_gives_the_same_rows(self):
        fake = types.ModuleType("fake_loader")
        fake.get_final_ledger = demo_ledger.get_final_ledger          # no scan_ledger: the cached path
        sys.modules["fake_loader"] = fake
        lazy, _ = ledger.get_ledger_slice(self.START, self.END, self.CLIENTS, TRADING)
        with self.settings(LEDGER_MODULE="fake_loader"):
            cached, _ = ledger.get_ledger_slice(self.START, self.END, self.CLIENTS, TRADING)
        order = ["ClientID", "CreatedAt", "LedgerType", "Debit", "Credit"]
        pd.testing.assert_frame_equal(lazy.sort_values(order).reset_index(drop=True),
                                      cached[order].sort_values(order).reset_index(drop=True), check_dtype=False)
```

Tool	What it does	Why here
`@override_settings(...) on a class`	Applies to every test in it	All ledger tests use the demo
`types.ModuleType('name')`	An empty module object	A fake loader with only what the test needs
`sys.modules['name'] = fake`	Registers it as already imported	importlib.import_module finds it
`sys.modules.pop('name', None)`	Removes it, no error if absent	The next test can't see the fake
`pd.testing.assert_frame_equal(a, b)`	Fails with a diff if tables differ	Two paths, same rows
`sort_values(...).reset_index(drop=True)`	Same order, fresh index	Compares content, not row order

Check yourself
Call get_ledger_slice through two different fake loaders (both without scan_ledger) without clear_cache() in between: the second returns the **first** loader's rows (checked). The cache doesn't record which loader filled it. That's harmless in production, where settings don't change while the server runs, and it's exactly the leak tearDown prevents in tests.

SimpleTestCase or TestCase?
`SimpleTestCase` refuses database access, so it's lighter, and right for pure pandas and Excel tests. `TestCase` gives a test database and rolls back after every test; use it whenever a test creates users or runs, or goes through views.

Part 4. Does a test notice? Mutation testing

A test that passes on broken code proves nothing. The check: **make one small, realistic mistake in the code, run the tests, and see which one fails**. These five were run against a copy of the project:

Mistake made on purpose	File	Tests that failed
(created < end) changed to (created <= end)	commission/services.py	test_client_pnl, test_broker_sheet
LedgerType no longer stripped and upper-cased	commission/services.py	test_client_pnl, test_broker_sheet
client_sheet merge how='left' changed to 'inner'	commission/services.py	test_broker_sheet
TradeDate no longer blank on non-BILL rows	activity/services.py	test_client_activity, test_manager_summary
Header spaces no longer stripped in read_csv	core/readers.py	**none**: every test still passed

The gap the last row found
The project's tests never upload a file whose headers have spaces, so nothing protects that line. The fix is a test, and it's checkpoint 1 below. With it added, the same mistake fails with `ValueError: The file is missing columns: Client ID` (checked).

How to run a mutation yourself

1. Make sure everything passes: `python manage.py test`.
2. Change one line in the way a tired person might (an operator, a merge type, a missing cleanup).
3. Run the tests. Note which fail. If none do, write the test that would have.
4. Undo the change (Ctrl+Z or `git checkout -- <file>`) and run the tests again.

Test first, when you fix a bug

On 4 Nov the broker workbook showed NBP as 0, because the ledger was filtered to BILL before NBP was looked for. The habit that stops a bug returning: **write a test that fails because of the bug, then fix the code until it passes**. The NBP row in the tiny data (Step 2) is exactly that test's input.

Part 5. Checkpoints

1. Close the gap: a test for header cleaning

Add this to `core/tests.py`. It passes on the project's code and fails if `df.columns = df.columns.str.strip()` is removed.

```
from io import BytesIO

from django.test import SimpleTestCase

from core.readers import read_csv


class ReadCsvTests(SimpleTestCase):
    def test_headers_and_values_are_stripped(self):
        df = read_csv(BytesIO(b" Client ID ,Name\n C1 ,x\n"), required=["Client ID"], text_columns=["Client ID"])
        self.assertEqual(df.loc[0, "Client ID"], "C1")
```

2. A rule activity doesn't test yet: a fully cancelled withdrawal

```
from django.test import SimpleTestCase
import pandas as pd

from activity import services


class CancelledWithdrawalTests(SimpleTestCase):
    def test_cancelled_withdrawal_nets_to_zero(self):
        led = pd.DataFrame({"ClientID": ["A", "A"],
                            "CreatedAt": pd.to_datetime(["2026-08-06 10:00", "2026-08-07 10:00"]),
                            "LedgerType": ["WITHDRAW", "WITHDRAW CANCELLED"],
                            "Debit": [300.0, 0.0], "Credit": [0.0, 300.0]})
        out = services.client_activity(led, pd.DataFrame({"Client ID": ["A"]})).set_index("Client ID")
        self.assertEqual(out.loc["A", "Withdraw"], 0)              # 300 out, 300 back
        self.assertEqual(out.loc["A", "Status"], "Not Traded")     # money moved, no trades
```

3. Run the five mutations yourself

Reproduce Part 4's table on your machine. For the third mutation, read the failure message: it's a `KeyError: 'P2'`, because broker Y has disappeared from the result.

Explain it back

Answer each in one or two sentences, without looking. The answers follow.

1. Why does the tiny ledger contain 'bill ' and a blank Debit on purpose?
2. Why compare Commission with assertAlmostEqual but Net_Profit with assertEqual?
3. Why do view tests use reverse() instead of typing '/commission/new/'?
4. What would go wrong if setUp enabled override_settings and tearDown didn't disable it?
5. How does the fake loader get imported without a file?
6. When should a test class be SimpleTestCase rather than TestCase?
7. What does it mean if no test fails after you break a line on purpose?

Answers

1. Each one is a real-data problem; if the code stopped handling it, a test would fail.
2. Commission comes from a multiplication and can carry float noise; the net is a sum of round numbers.
3. URLs come from names; if an address changes, the tests still find the right page.
4. The temporary media folder and demo ledger would leak into later tests, which could then write or read the wrong place.
5. It's a types.ModuleType object placed in sys.modules, which import checks before looking for files.
6. When it doesn't touch the database: pure pandas, Excel bytes, helpers.
7. That line isn't protected by any test; write the test that would have caught it.

Appendix. The test files, with line numbers

commission/tests.py

```
  1  import shutil
  2  import tempfile
  3  from pathlib import Path
  4
  5  import pandas as pd
  6  from django.contrib.auth import get_user_model
  7  from django.core.files.uploadedfile import SimpleUploadedFile
  8  from django.test import TestCase, override_settings
  9  from django.urls import reverse
 10
 11  from . import services
 12  from .models import CommissionRun
 13
 14  SAMPLE_CSV = Path(__file__).parent / "sample_data" / "brokers_sample.csv"
 15
 16
 17  class ServiceTests(TestCase):
 18      """The calculation alone, on a table small enough to check by hand."""
 19
 20      def setUp(self):
 21          self.led = pd.DataFrame({
 22              "ClientID":   ["A", "A", "A", "B", "B"],
 23              "CreatedAt":  pd.to_datetime(["2026-08-02", "2026-08-03", "2026-08-04", "2026-08-05", "2026-09-01"]),
 24              "LedgerType": ["BILL", "bill ", "NEGATIVE BALANCE PROTECTION", "BILL", "BILL"],
 25              "Debit":      [100.0, None, 0.0, 0.0, 500.0],
 26              "Credit":     [0.0, 30.0, 10.0, 80.0, 0.0],
 27          })
 28          self.brokers = pd.DataFrame({"Broker Name": ["X", "X", "Y"], "Broker Client ID": ["P1", "P1", "P2"],
 29                                       "Broker Referral Count": [2, 2, 1],
 30                                       "Referred User Client ID": ["A", "B", "C"]})
 31
 32      def test_client_pnl(self):
 33          start, end = services.month_range("2026-08")
 34          pnl = services.client_pnl(self.led, ["A", "B"], start, end).set_index("ClientID")
 35          self.assertEqual(pnl.loc["A", "Net_Profit"], -60)          # 30 - 100 + 10
 36          self.assertAlmostEqual(pnl.loc["A", "Commission"], 3.0)    # 5% of 60
 37          self.assertEqual(pnl.loc["B", "Net_Profit"], 80)           # the September loss is outside August
 38
 39      def test_broker_sheet(self):
 40          report = services.build_commission(self.led, self.brokers, "2026-08")
 41          brokers = report["brokers"].set_index("Parent")
 42          self.assertEqual(brokers.loc["P1", "Difference"], 10)      # (30 + 80) - 100
 43          self.assertEqual(brokers.loc["P1", "Loss Status"], "NO LOSS")
 44          self.assertEqual(brokers.loc["P2", "No of Children"], 1)   # C never traded: zeros, still listed
 45          self.assertEqual(report["summary"]["clients"], 3)
 46
 47      def test_missing_column_is_a_clear_error(self):
 48          with self.assertRaisesMessage(ValueError, "missing columns"):
 49              services.client_pnl(self.led.drop(columns="Credit"), ["A"], *services.month_range("2026-08"))
 50
 51
 52  class ViewTests(TestCase):
 53      """The whole flow through the web pages, with the demo ledger."""
 54
 55      def setUp(self):
 56          self.media = tempfile.mkdtemp()
 57          self.override = override_settings(MEDIA_ROOT=self.media, LEDGER_MODULE="core.demo_ledger")
 58          self.override.enable()
 59          self.user = get_user_model().objects.create_user("analyst", password="pass12345")
 60
 61      def tearDown(self):
 62          self.override.disable()
```

```
        shutil.rmtree(self.media, ignore_errors=True)

    def test_login_required(self):
        response = self.client.get(reverse("commission:run_list"))
        self.assertEqual(response.status_code, 302)
        self.assertIn(reverse("login"), response["Location"])

    def test_create_report(self):
        self.client.login(username="analyst", password="pass12345")
        upload = SimpleUploadedFile("brokers.csv", SAMPLE_CSV.read_bytes(), content_type="text/csv")
        response = self.client.post(reverse("commission:run_create"),
                                    {"month": "2026-08", "rate": "0.05", "broker_file": upload})
        run = CommissionRun.objects.get()
        self.assertRedirects(response, reverse("commission:run_detail", args=[run.pk]))
        self.assertEqual(run.brokers_count, 5)
        self.assertEqual(run.clients_count, 30)
        self.assertTrue(run.result_file.name.endswith(".xlsx"))

        detail = self.client.get(reverse("commission:run_detail", args=[run.pk]))
        self.assertContains(detail, "Demo Broker 1")
        download = self.client.get(reverse("commission:run_download", args=[run.pk]))
        self.assertEqual(download["Content-Disposition"], 'attachment; filename="Broker_Commission_2026-08.xlsx"')

    def test_bad_month_is_rejected(self):
        self.client.login(username="analyst", password="pass12345")
        upload = SimpleUploadedFile("brokers.csv", SAMPLE_CSV.read_bytes(), content_type="text/csv")
        response = self.client.post(reverse("commission:run_create"),
                                    {"month": "08-2026", "rate": "0.05", "broker_file": upload})
        self.assertContains(response, "Use the format YYYY-MM")
        self.assertEqual(CommissionRun.objects.count(), 0)
```

Lines	Block	Purpose
17-47	`ServiceTests`	Rules on a five-row ledger
52-92	`ViewTests`	Pages with the test client

activity/tests.py

```python
import shutil
import tempfile
from pathlib import Path

import pandas as pd
from django.contrib.auth import get_user_model
from django.core.files.uploadedfile import SimpleUploadedFile
from django.test import TestCase, override_settings
from django.urls import reverse

from . import services
from .models import ActivityRun

SAMPLE_CSV = Path(__file__).parent / "sample_data" / "clients_sample.csv"


class ServiceTests(TestCase):
    def setUp(self):
        self.led = pd.DataFrame({
            "ClientID":   ["A", "A", "A", "A", "A", "B"],
            "CreatedAt":  pd.to_datetime(["2026-08-02 09:00", "2026-08-02 15:00", "2026-08-05 10:00", "2026-08-06 10:00",
                                          "2026-08-07 10:00", "2026-08-09 10:00"]),
            "LedgerType": ["BILL", "BILL", "DEPOSIT", "WITHDRAW", "WITHDRAW CANCELLED", "DEPOSIT"],
            "Debit":      [10.0, 0.0, 0.0, 300.0, 0.0, 0.0],
            "Credit":     [0.0, 5.0, 1000.0, 0.0, 100.0, None],
        })
        self.clients = pd.DataFrame({"Client ID": ["A", "B", "C"], "Manager": ["m1", "m1", None]})

    def test_client_activity(self):
        out = services.client_activity(self.led, self.clients).set_index("Client ID")
        self.assertEqual(out.loc["A", "Bills"], 2)
        self.assertEqual(out.loc["A", "Trading Days"], 1)          # two bills on the same day
        self.assertEqual(out.loc["A", "Withdraw"], 200)            # 300 withdrawn, 100 cancelled
        self.assertEqual(out.loc["A", "Net Flow"], 800)
        self.assertEqual(out.loc["B", "Status"], "Not Traded")     # deposits only
        self.assertEqual(out.loc["C", "Deposit"], 0)               # no ledger rows at all

    def test_manager_summary(self):
        summary = services.manager_summary(services.client_activity(self.led, self.clients)).set_index("Manager")
        self.assertEqual(summary.loc["m1", "Total Clients"], 2)
        self.assertEqual(summary.loc["m1", "Traded %"], 50.0)
        self.assertEqual(summary.loc["Unassigned", "Total Clients"], 1)

    def test_no_manager_column_means_no_summary(self):
        self.assertIsNone(services.manager_summary(services.client_activity(self.led, self.clients[["Client ID"]])))


class ViewTests(TestCase):
    def setUp(self):
        self.media = tempfile.mkdtemp()
        self.override = override_settings(MEDIA_ROOT=self.media, LEDGER_MODULE="core.demo_ledger")
        self.override.enable()
        get_user_model().objects.create_user("analyst", password="pass12345")
        self.client.login(username="analyst", password="pass12345")

    def tearDown(self):
        self.override.disable()
        shutil.rmtree(self.media, ignore_errors=True)

    def test_create_report(self):
        upload = SimpleUploadedFile("clients.csv", SAMPLE_CSV.read_bytes(), content_type="text/csv")
        response = self.client.post(reverse("activity:run_create"), {"month": "2026-09", "client_file": upload})
        run = ActivityRun.objects.get()
        self.assertRedirects(response, reverse("activity:run_detail", args=[run.pk]))
        self.assertEqual(run.clients_count, 32)
        self.assertLessEqual(run.traded_count, 30)                 # the two extra clients have no rows
        detail = self.client.get(reverse("activity:run_detail", args=[run.pk]))
        self.assertContains(detail, "Per manager and agent")

    def test_missing_client_id_column(self):
        upload = SimpleUploadedFile("clients.csv", b"ID,Name\nS200000,x\n", content_type="text/csv")
        response = self.client.post(reverse("activity:run_create"), {"month": "2026-09", "client_file": upload})
        self.assertContains(response, "missing columns: Client ID")
        self.assertEqual(ActivityRun.objects.count(), 0)
```

Lines	Block	Purpose
17-44	`ServiceTests`	Activity rules
48-74	`ViewTests`	Upload, detail, missing column

core/tests.py

```
import sys
import types

import pandas as pd
import polars as pl
from django.contrib.auth import get_user_model
from django.test import TestCase, override_settings
from django.urls import reverse

from . import demo_ledger, ledger
from .excel import SheetSpec, workbook_bytes
from .periods import month_range, previous_month
from .reports import REPORTS

TRADING = ["BILL", "NEGATIVE BALANCE PROTECTION"]


class PeriodTests(TestCase):
    def test_month_range_is_half_open(self):
        self.assertEqual(month_range("2026-12"), (pd.Timestamp("2026-12-01"), pd.Timestamp("2027-01-01")))

    def test_previous_month_crosses_the_year(self):
        self.assertEqual(previous_month("2026-01-15"), "2025-12")


class ExcelTests(TestCase):
    def test_sheets_with_and_without_bands(self):
        df = pd.DataFrame({"A": ["x"], "B": [1.5]})
        data = workbook_bytes([SheetSpec("Banded", df, bands=[("Band", 1, 2, "1F3864")], money_cols=(2,)),
                               SheetSpec("Plain", df, money_cols=(2,))])
        banded = pd.read_excel(pd.io.common.BytesIO(data), sheet_name="Banded", header=1)
        plain = pd.read_excel(pd.io.common.BytesIO(data), sheet_name="Plain")
        self.assertEqual(list(banded.columns), ["A", "B"])
        self.assertEqual(list(plain.columns), ["A", "B"])


class PageTests(TestCase):
    def test_login_page_renders(self):
        response = self.client.get(reverse("login"))
        self.assertContains(response, "Log in")


class HomeTests(TestCase):
    def test_home_lists_every_registered_report(self):
        get_user_model().objects.create_user("analyst", password="pass12345")
        self.client.login(username="analyst", password="pass12345")
        response = self.client.get(reverse("core:home"))
        for report in REPORTS:
            self.assertContains(response, report.title)
        self.assertGreaterEqual(len(REPORTS), 2)


@override_settings(LEDGER_MODULE="core.demo_ledger")
class LedgerSliceTests(TestCase):
    """Reading only the rows a report needs."""

    START, END = pd.Timestamp("2026-08-01"), pd.Timestamp("2026-09-01")
    CLIENTS = demo_ledger.DEMO_CLIENTS[:3]

    def tearDown(self):
        ledger.clear_cache()
        sys.modules.pop("fake_loader", None)
```

```
63
64      def test_lazy_slice_keeps_only_what_was_asked(self):
65          rows, notes = ledger.get_ledger_slice(self.START, self.END, clients=self.CLIENTS, types=TRADING)
66          self.assertGreater(len(rows), 0)
67          self.assertTrue(rows["CreatedAt"].between(self.START, self.END, inclusive="left").all())
68          self.assertTrue(set(rows["ClientID"]) <= set(self.CLIENTS))
69          self.assertTrue(set(rows["LedgerType"]) <= set(TRADING))
70          self.assertEqual(notes, [])
71
72      def test_loader_without_scan_ledger_gives_the_same_rows(self):
73          fake = types.ModuleType("fake_loader")
74          fake.get_final_ledger = demo_ledger.get_final_ledger          # no scan_ledger: the cached path
75          sys.modules["fake_loader"] = fake
76          lazy, _ = ledger.get_ledger_slice(self.START, self.END, self.CLIENTS, TRADING)
77          with self.settings(LEDGER_MODULE="fake_loader"):
78              cached, _ = ledger.get_ledger_slice(self.START, self.END, self.CLIENTS, TRADING)
79          order = ["ClientID", "CreatedAt", "LedgerType", "Debit", "Credit"]
80          pd.testing.assert_frame_equal(lazy.sort_values(order).reset_index(drop=True),
81                                        cached[order].sort_values(order).reset_index(drop=True), check_dtype=False)
82
83      def test_unreadable_dates_are_reported(self):
84          fake = types.ModuleType("fake_loader")
85          frame = pl.DataFrame({"ClientID": ["A", "A"],
86                                "CreatedAt": [pd.Timestamp("2026-08-05").to_pydatetime(), None],
87                                "LedgerType": ["BILL", "BILL"], "Debit": [10.0, 5.0], "Credit": [0.0, 0.0]})
88          fake.scan_ledger = lambda columns=None, only_new_appends=False: frame.lazy()
89          sys.modules["fake_loader"] = fake
90          with self.settings(LEDGER_MODULE="fake_loader"):
91              rows, notes = ledger.get_ledger_slice(self.START, self.END, clients=["A"])
92          self.assertEqual(len(rows), 1)
93          self.assertIn("1 ledger rows have an unreadable CreatedAt", notes[0])
```

Lines	Block	Purpose
18-22	`PeriodTests`	Month helpers
26-34	`ExcelTests`	Banded and plain sheets
37-40	`PageTests`	The login page renders
43-50	`HomeTests`	Every report is listed
54-93	`LedgerSliceTests`	Lazy path, fallback, unreadable dates